Task 1: Problem Definition and Scope

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Define the problem, scope, users, objectives, constraints, and measurable success criteria for the Puppet-Provisioned Delivery Promise Calculator.

2. Problem Statement

The project is a web application for managing delivery records and calculating promised delivery dates. The larger DevOps objective is to make the application buildable, testable, deployable, configurable, and recoverable through automation.

3. Target Users

• Customer: view and search delivery records.

• Operations staff: manage operational delivery status.

• Administrator: manage application data and access.

• Development/DevOps team: maintains source, CI/CD, containers and configuration.

4. MVP Scope

ID	Feature
F01	User login / role context
F02	Role-based access
F03	Create delivery
F04	View delivery records
F05	Update delivery
F06	Search by customer/product
F07	Promised-date calculation
F08	Status display
F09	Authorized status update
F10	Delivery details
F11	Dashboard counts
F12	Filtering/search
F13	Validation
F14	Error handling
F15	Health/verification support

5. Constraints

• Java 21 and Spring Boot with Maven.

• SQLite is used for persistence.

• Jenkins uses port 8080; the application uses port 8765.

• Docker uses host port 8766 mapped to container port 8765.

• The workflow must run on the Windows development machine.

6. Success Criteria

• Maven build succeeds.

• Selenium tests pass.
• Jenkins executes CI/CD.

• Docker image/container runs successfully.

• Puppet establishes and restores desired configuration.

• Browser can access the deployed application.

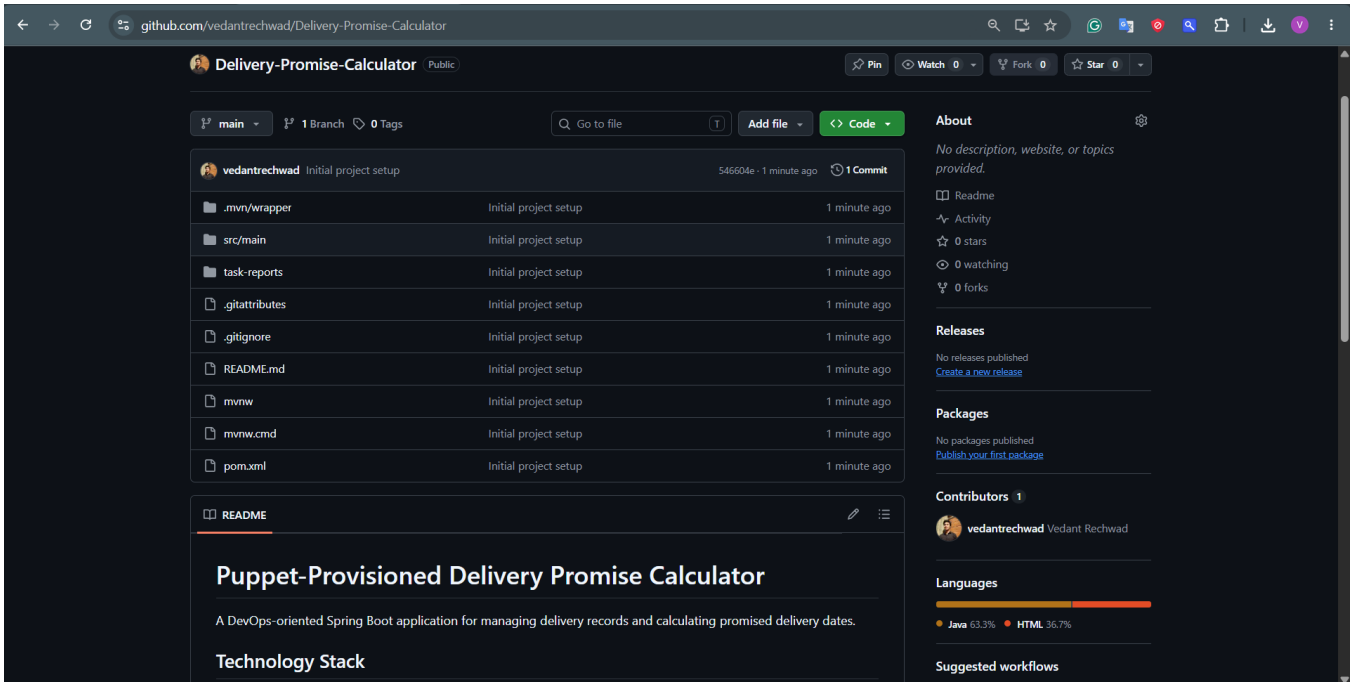
7. Result

The project scope was defined as a delivery-management application combined with a complete DevOps automation workflow.

Task 1 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Project GitHub repository and README showing the Delivery Promise Calculator project scope.



Task 2: Agile Planning and DevOps Workflow

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Plan the work using user stories, acceptance criteria, a task backlog, Definition of Done, and a DevOps lifecycle.

2. User Stories

ID	User story
US-01	User can access the delivery application.
US-02	User receives access according to role.
US-03	Authorized user can create a delivery.
US-04	User can view delivery records.
US-05	Authorized user can update delivery information.

US-06	User can search customer/product.
US-07	System calculates promised date.
US-08	System displays status.
US-09	Authorized roles can update status.
US-10	User can view details.
US-11	Dashboard shows counts.
US-12	Required input is validated.
US-13	Invalid input is handled clearly.
US-14	Application can be built/tested automatically.
US-15	Application can be deployed/configured automatically.

3. Task Backlog

Task	Work item	Final
1	Problem definition	Done
2	Agile planning	Done
3	Requirements/setup	Done
4	Git/GitHub	Done
5	Feature branching	Done
6	MVP/collaboration	Done
7	Jenkins CI	Done
8	Pipeline/deployment	Done
9	Selenium local	Done
10	Continuous testing	Done
11	Docker lifecycle	Done
12	Jenkins-Docker CD	Done
13	Puppet configuration	Done
14	Provisioning/reliability	Done
15	Final release/viva	Done

4. Definition of Done

• Feature implemented and tested.

• Git change committed and reviewed where applicable.

• Build passes.

• Relevant automated tests pass.

• Deployment/configuration behavior verified.

• Evidence recorded for the task.

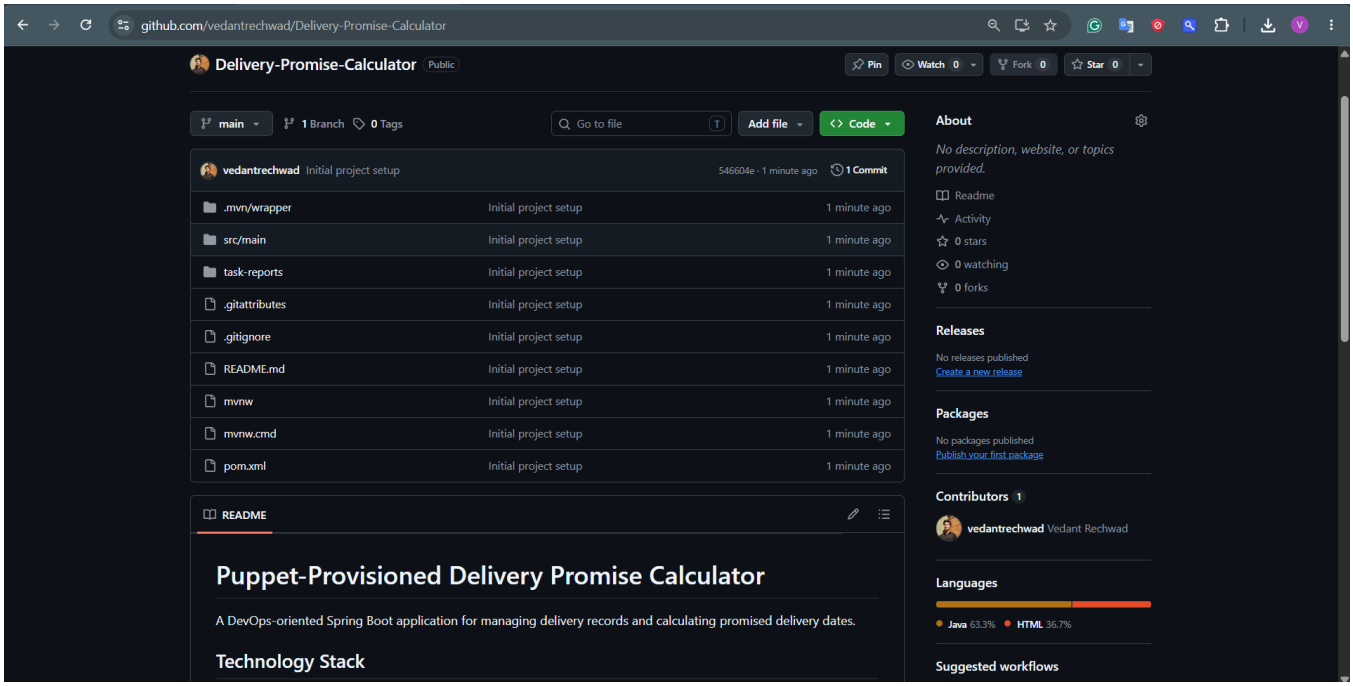
5. DevOps Lifecycle

Plan → Code → Build → Test → Release → Deploy → Configure/Operate → Verify. The project maps these phases to GitHub, Maven, Jenkins, Selenium, Docker, and Puppet.

Task 2 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. GitHub repository used as the project collaboration baseline.



Task 3: Requirements Architecture and Technology Setup

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Set up the Spring Boot application, dependencies, database configuration, project structure, and initial delivery-management architecture.

2. Spring Initializr

• Build: Maven

• Language: Java

• Spring Boot: 4.1.1

• Group: com.deliverypromise

• Artifact: delivery-promise-calculator

• Package: com.deliverypromise.calculator

• Packaging: Jar

• Java: 21

3. Dependencies

Spring Web/WebMVC, Thymeleaf, Spring Data JPA, Validation, DevTools, SQLite JDBC, and Hibernate Community Dialects.

4. SQLite POM additions

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.53.4.0</version>
```

```
</dependency>
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-community-dialects</artifactId>
</dependency>
```

5. application.properties

```
spring.application.name=delivery-promise-calculator
spring.datasource.url=jdbc:sqlite:delivery_promise.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.jdbc.use_get_generated_keys=false
server.port=8765
```

6. Project Structure

```
src/main/java/com/deliverypromise/calculator/
■■■ DeliveryPromiseCalculatorApplication.java
■■■ HomeController.java
■■■ controller/DeliveryController.java
■■■ model/Delivery.java
■■■ repository/DeliveryRepository.java
■■■ service/DeliveryService.java

src/main/resources/
■■■ application.properties
■■■ templates/
    ■■■ index.html
    ■■■ deliveries.html
```

7. Application Logic

The Delivery entity stores customerName, deliveryAddress, productName, deliveryDays, promisedDate, and status. The service calculates promisedDate as LocalDate.now().plusDays(deliveryDays), defaults status to PENDING, and supports customer/product search and status updates.

8. Commands

```
mvn clean test
mvn spring-boot:run
# Browser verification
http://localhost:8765/
```

9. Result

The application built and ran successfully, with SQLite persistence and the initial delivery-management pages available.

Task 3 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Spring Initializr configuration used to create the Java/Spring Boot project.



Figure 2. Java/Maven environment verification.

```
PS C:\Users\vedan> java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS, mi
xed mode, sharing)
PS C:\Users\vedan> mvn -version
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: C:\DevTools\apache-maven-3.9.16
Java version: 21.0.12, vendor: Eclipse Adoptium, runtime: C:\Progra
m Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "win
dows"
```

Figure 3. Running Delivery Promise Calculator on localhost:8765.

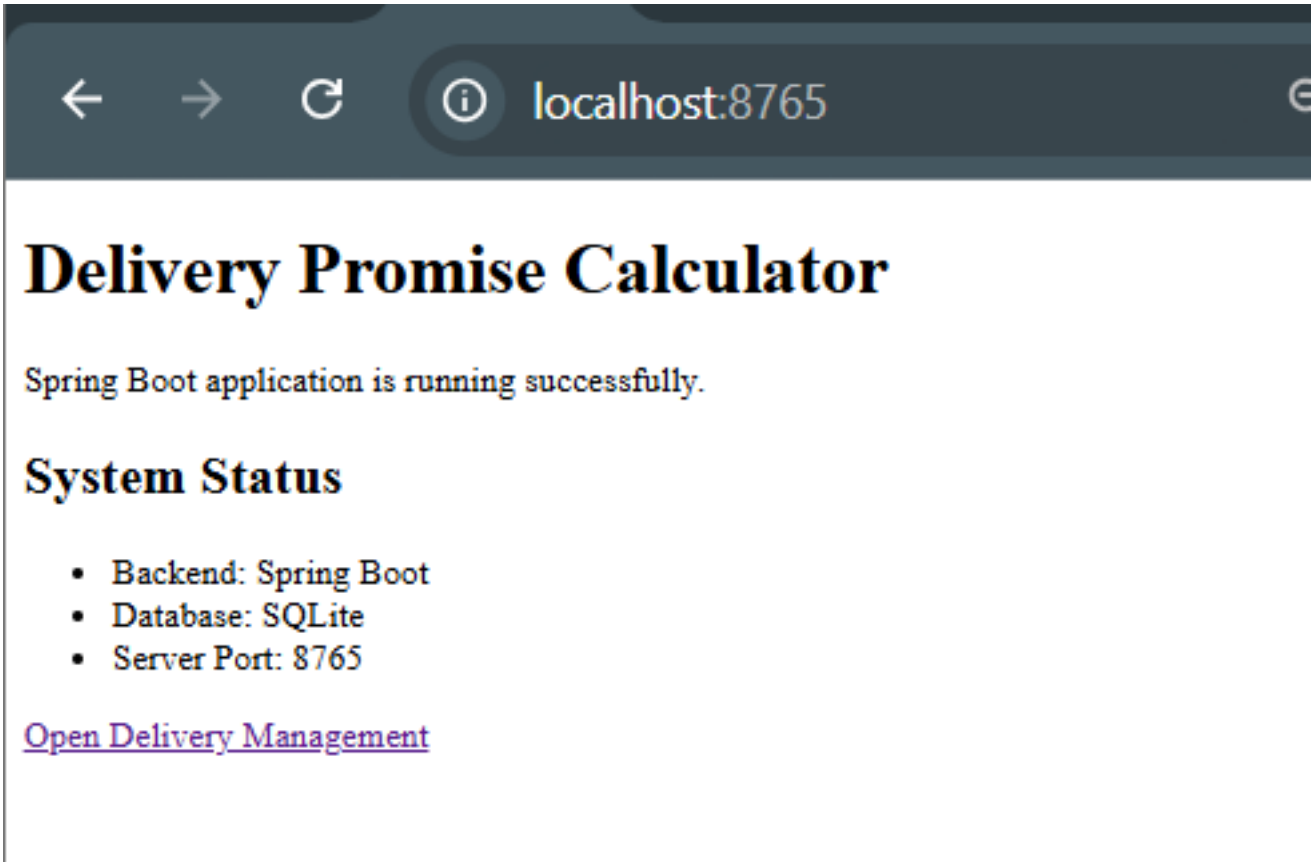
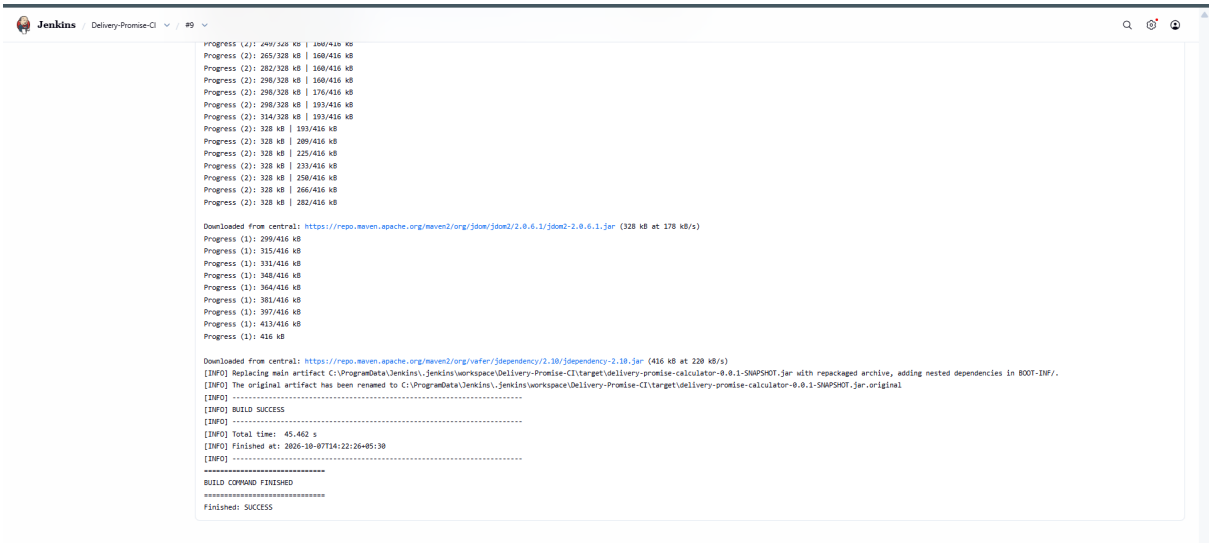


Figure 4. Maven build/package execution and successful build output.



Task 4: Git GitHub Repository Initialization

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Initialize Git, connect the project to GitHub, ignore local/generated files, and establish main as the integration branch.

2. .gitignore

```
### Application database ###
*.db
delivery_promise.db
```

3. Commands Used

```
git init
git branch -M main
git remote add origin https://github.com/vedantrechwad/Delivery-Promise-Calculator.git
git add .
git commit -m "Initialize Delivery Promise Calculator project"
git push -u origin main
```

4. Branching Policy

• main is the integration branch.

• Feature changes are developed on feature branches.

• Branches are pushed to GitHub.

• Pull requests are used to merge reviewed work into main.

5. Why Ignore the Database?

SQLite database files represent local runtime state rather than source code. Ignoring *.db prevents accidental database commits.

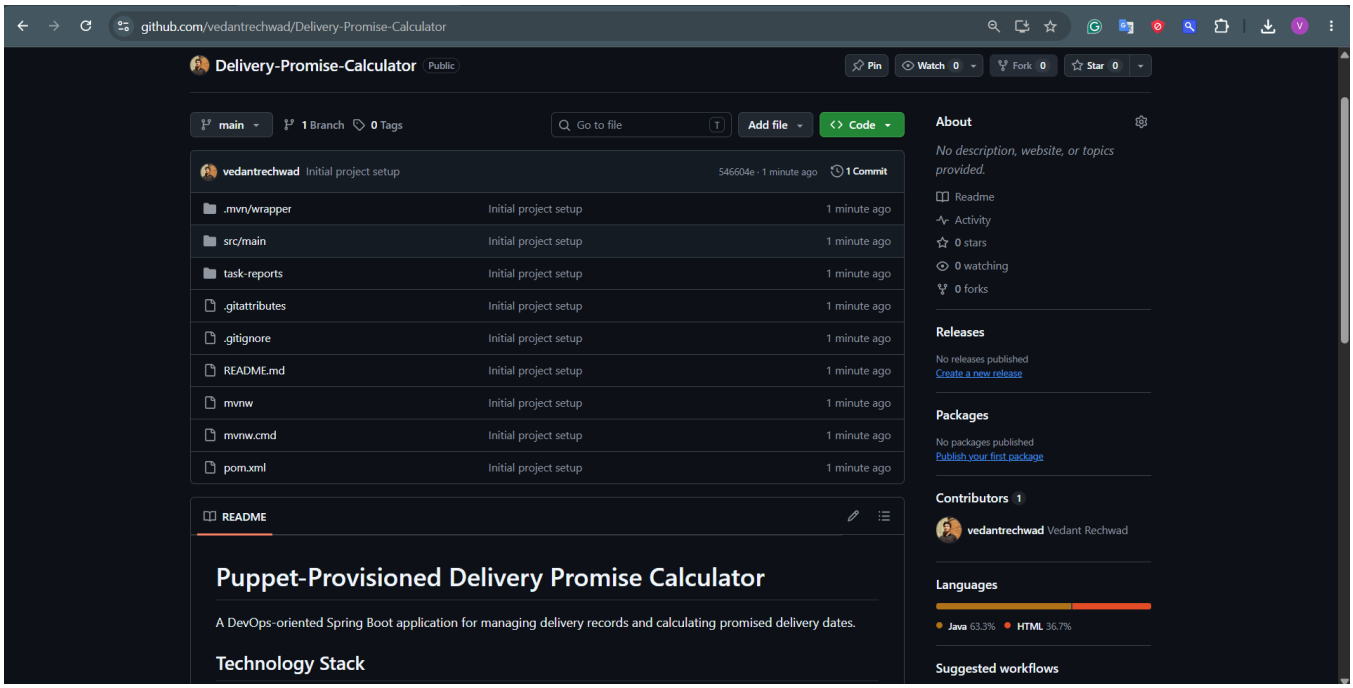
6. Result

The Delivery Promise Calculator became a Git/GitHub-managed project with a clean source-control baseline.

Task 4 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. GitHub repository on the main branch with project files and README.



Task 5: Feature Development with Branching

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Implement input validation on an isolated feature branch and merge it into main through a pull request.

2. Feature

Required delivery fields were validated and deliveryDays was constrained to a minimum of 1.

3. Validation Code

```
@NotBlank
private String customerName;

@NotBlank
private String deliveryAddress;

@NotBlank
private String productName;

@Min(1)
private int deliveryDays;
```

4. Controller Integration

```
@PostMapping("/create")
public String createDelivery(
        @Valid @ModelAttribute Delivery delivery,
        BindingResult result,
        ...) {
    if (result.hasErrors()) {
        return "deliveries";
    }
    // save through service
}
```

5. Git Commands

```
git checkout -b feature/delivery-management
git add .
git commit -m "Add delivery input validation"
git push -u origin feature/delivery-management
```

6. Pull Request

The PR was titled “Add delivery input validation” and was merged into main.

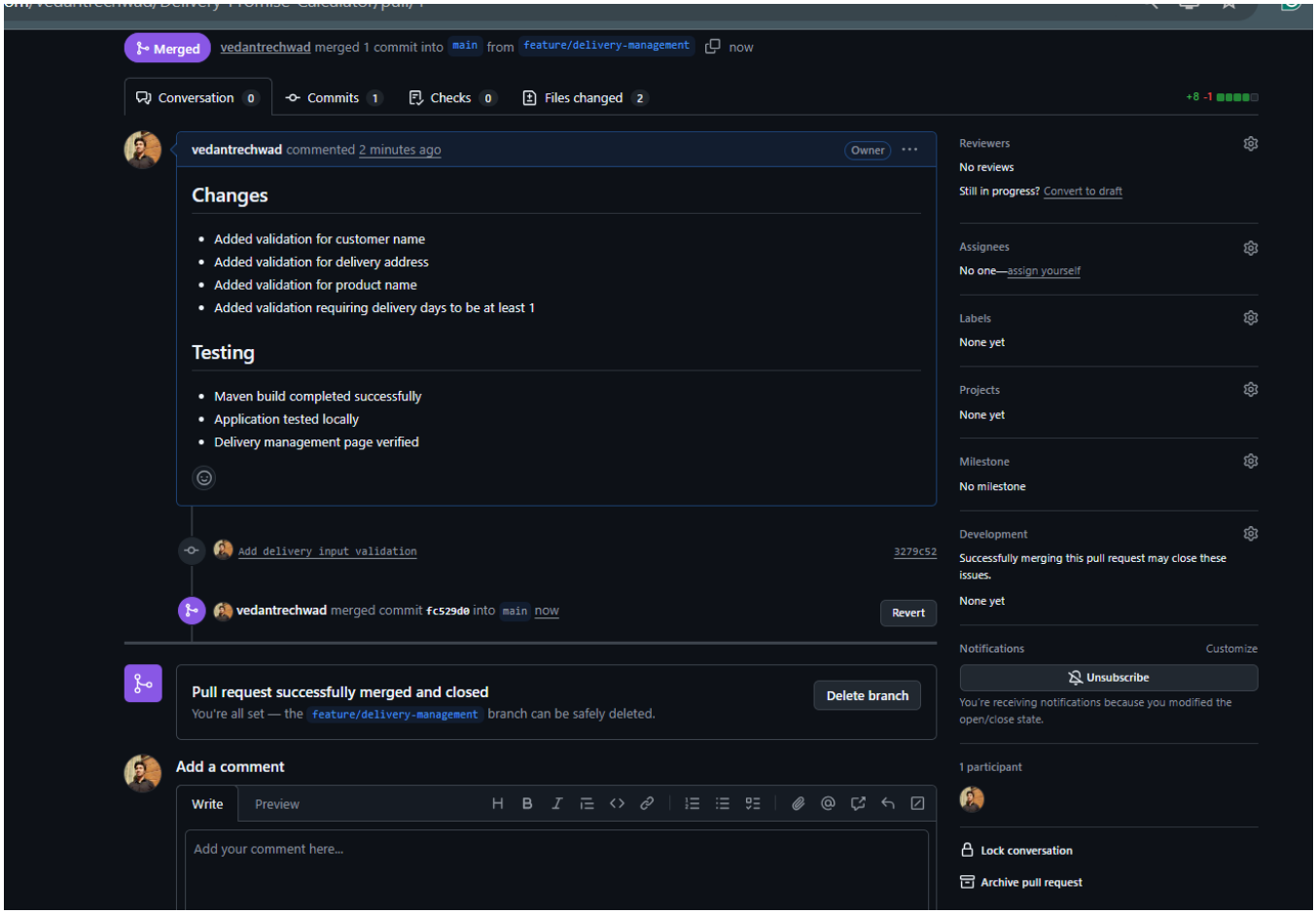
7. Result

Validation was implemented independently, reviewed through GitHub, and integrated without bypassing the branching workflow.

Task 5 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. GitHub pull request showing delivery validation changes merged into main.



Task 6: MVP Completion and Git Collaboration

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Complete the MVP, implement role-based behavior/dashboard features, and demonstrate real Git collaboration including conflict resolution.

2. MVP Components

```
RoleController
DeliveryController
DashboardController
dashboard.html
deliveries.html
```

3. Role Behavior

Role	Access
CUSTOMER	View/search delivery records
OPERATIONS	View records and update status
ADMIN	Administrative access and status operations

4. Feature Branch

```
git checkout -b feature/mvp-dashboard
git add .
git commit -m "Add MVP dashboard and role workflow"
git push -u origin feature/mvp-dashboard
```

5. README / MVP Documentation

The README was extended with MVP features, role workflow, dashboard behavior, and Git collaboration notes.

6. Merge Conflict Demonstration

A README conflict was intentionally demonstrated. Feature commit aab7900 and main commit 6f7790e conflicted. The resolution was committed as 6b25eec with message “Resolve README collaboration conflict”.

7. Conflict Resolution Commands

```
git checkout main
git pull
# resolve conflict in README.md
git add README.md
git commit -m "Resolve README collaboration conflict"
git push origin main
```

8. Result

The MVP and role-based workflow were completed, and collaborative Git practices were demonstrated through a real conflict and resolution.

Task 6 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. ADMIN role view showing delivery record and authorized status controls.

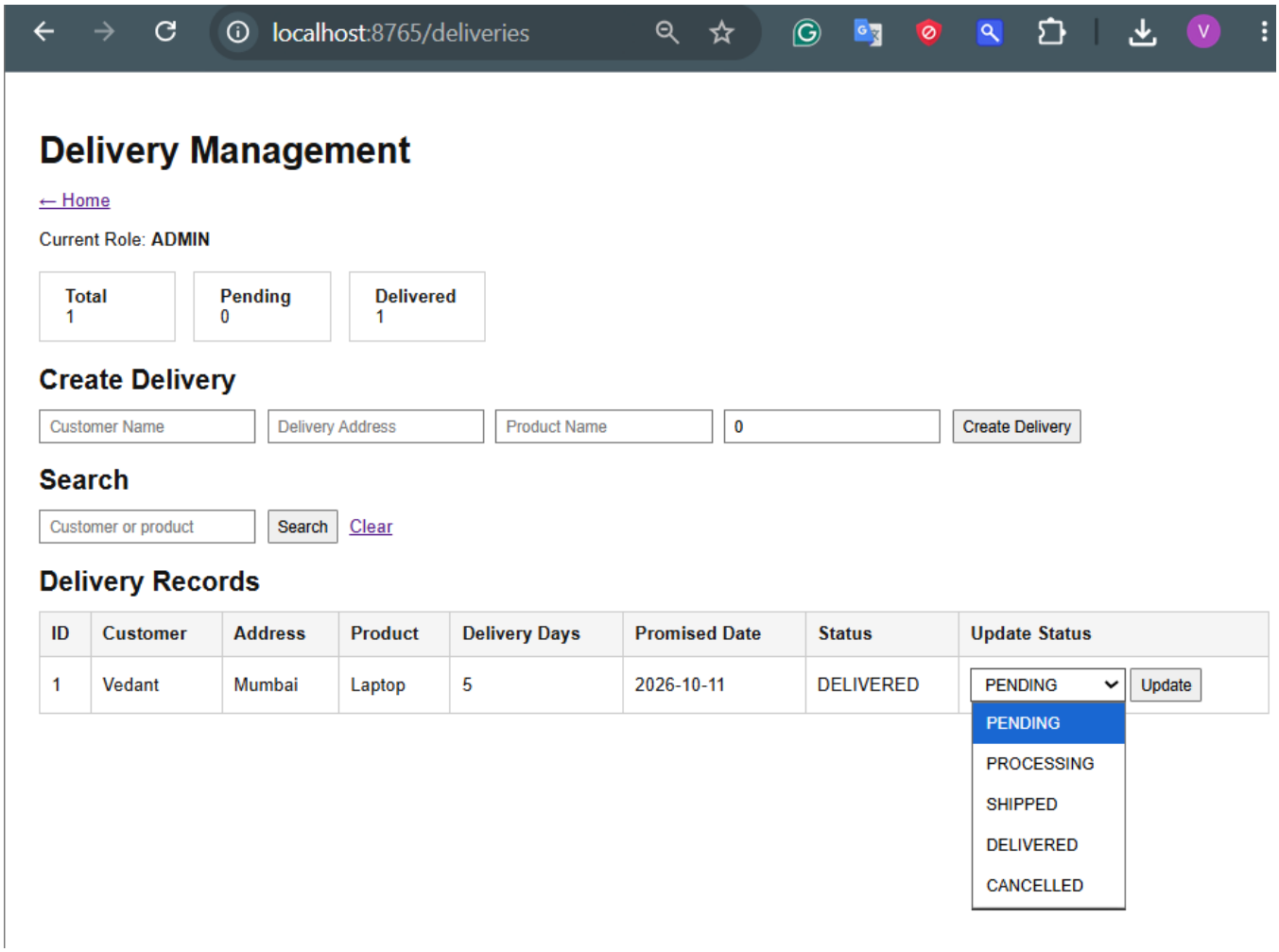
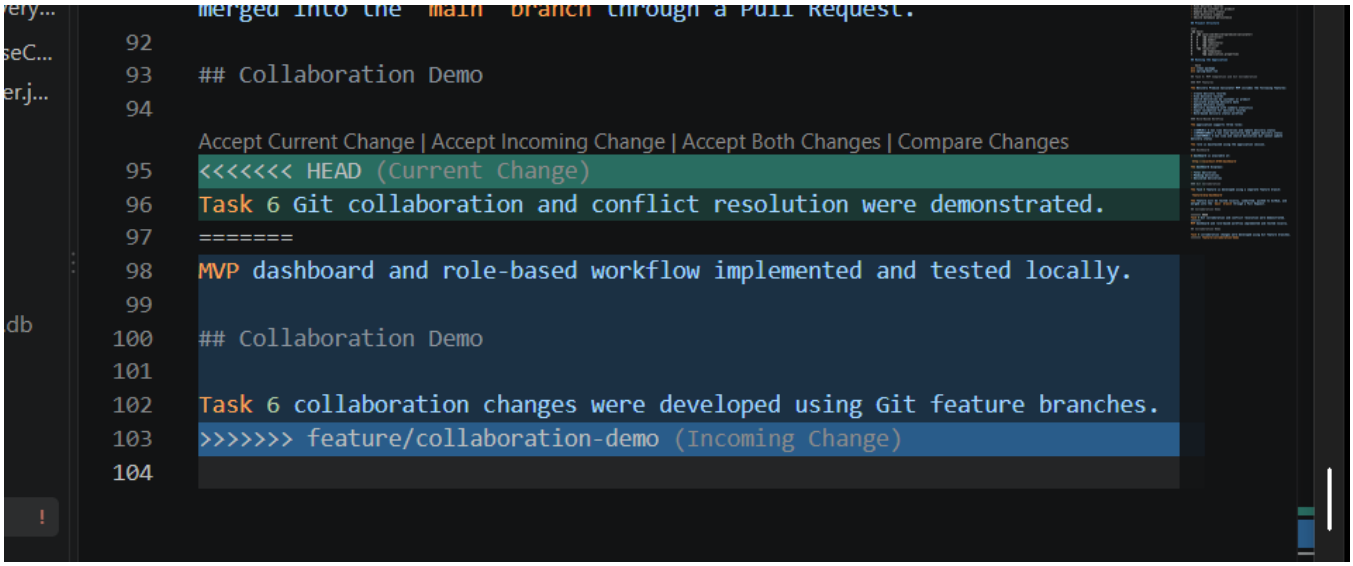


Figure 2. Git merge-conflict demonstration in the README during collaboration work.



Task 7: Jenkins Installation and CI Job

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Install/configure Jenkins and create a CI job that checks out the GitHub project and performs the Maven build/test/package process.

2. Environment

• Jenkins: http://localhost:8080

• OpenJDK: 21.0.12

• Git: 2.52.0

• Maven: 3.9.16

• Workspace: C:\ProgramData\Jenkins\.jenkins\workspace\Delivery-Promise-CI

3. Jenkins Job

• Job: Delivery-Promise-CI

• Type: Freestyle

• SCM: GitHub main branch

• Build: Maven clean test package

4. Maven PATH Problem

Jenkins initially could not find Maven from the service environment. The build was fixed by using the absolute Maven executable path.

5. Commands

```
C:\DevTools\apache-maven-3.9.16\bin\mvn.cmd -version
C:\DevTools\apache-maven-3.9.16\bin\mvn.cmd clean test package
```

6. CI Purpose

The job makes source checkout, compilation, tests, and packaging repeatable and independent of a developer's interactive shell.

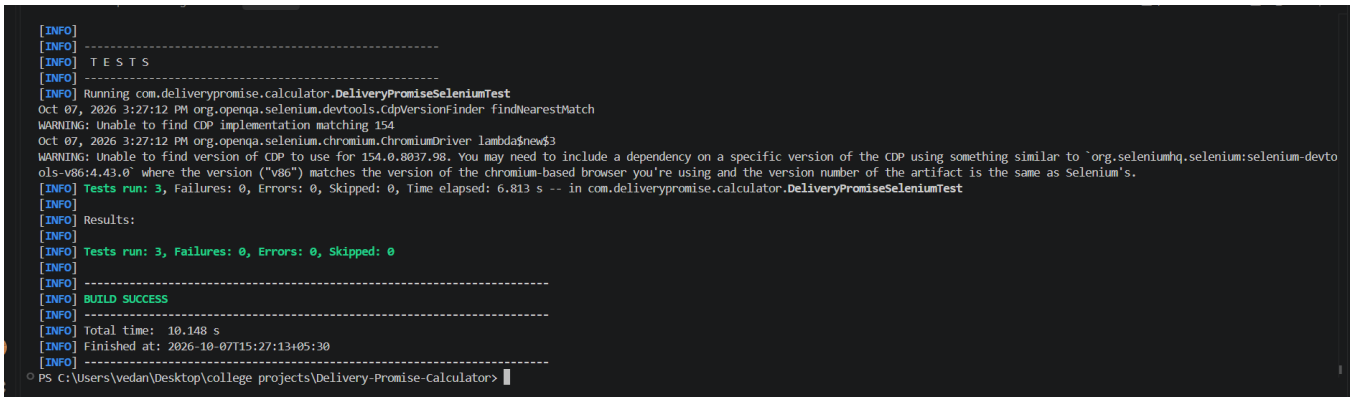
7. Result

The Jenkins CI job completed successfully after correcting the Maven path.

Task 7 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Maven test execution showing successful Selenium tests and BUILD SUCCESS.



Task 8: Pipeline as Code and Server Deployment

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Automate application build/deployment through a Jenkinsfile and deploy the Spring Boot JAR to the Windows environment.

2. Deployment

```
C:\DeliveryPromise\delivery-promise-calculator-0.0.1-SNAPSHOT.jar
Application: 8765
Jenkins: 8080
```

3. Jenkinsfile Core

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build and Test') {
            steps {
                bat '"C:\DevTools\apache-maven-3.9.16\bin\mvn.cmd" clean test package'
            }
        }
    }
}
```

4. Jenkins Process-Cleanup Issue

A background Spring Boot process was initially terminated by Jenkins process cleanup. The deployment was corrected by setting Jenkins process-cookie variables and launching the application as a detached process.

5. Deployment Command

```
set "JENKINS_SERVER_COOKIE=dontKillMe"
set "JENKINS_NODE_COOKIE=dontKillMe"
start "Delivery Promise Calculator" /B cmd /c "java -jar "C:\DeliveryPromise\delivery-promise-calculator-0.0.1-SNAPSHOT.jar"
```

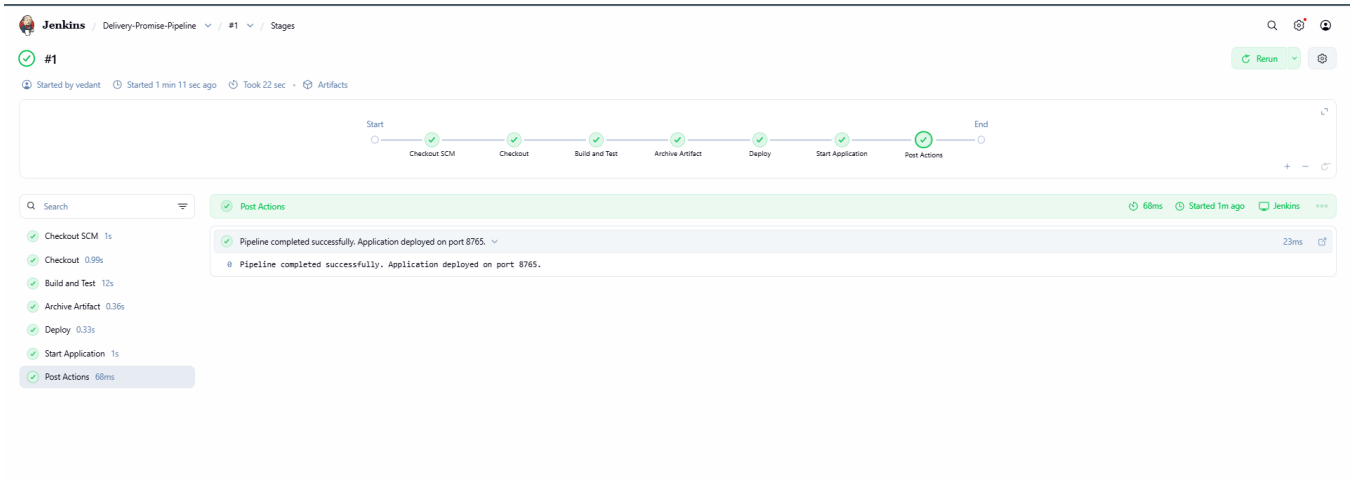
6. Result

The pipeline successfully built the application and deployed the JAR on port 8765.

Task 8 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Jenkins pipeline showing successful checkout, build/test, artifact, deployment and application-start stages.



Task 9: Selenium Test Design and Local Execution

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Create browser-level functional tests for the delivery application and execute them locally.

2. Environment

Item	Value
Java	21
Maven	3.9.16
Automation	Selenium WebDriver
Browser	Chrome
Test framework	JUnit 5
Application	http://localhost:8765

3. Test Class

The Selenium class is DeliveryPromiseSeleniumTest under src/test/java/com/deliverypromise/calculator/. It covers page load, delivery creation, and search of the created delivery.

4. Test Cases

Test	Verification
Page load	Delivery Management page opens.

Create delivery	A delivery can be submitted through the UI.
Search	The created delivery can be located through search.

5. Execution Command

```
mvn clean test -Dtest=DeliveryPromiseSeleniumTest
```

6. Result

3 tests executed with 0 failures, 0 errors, and 0 skipped tests.

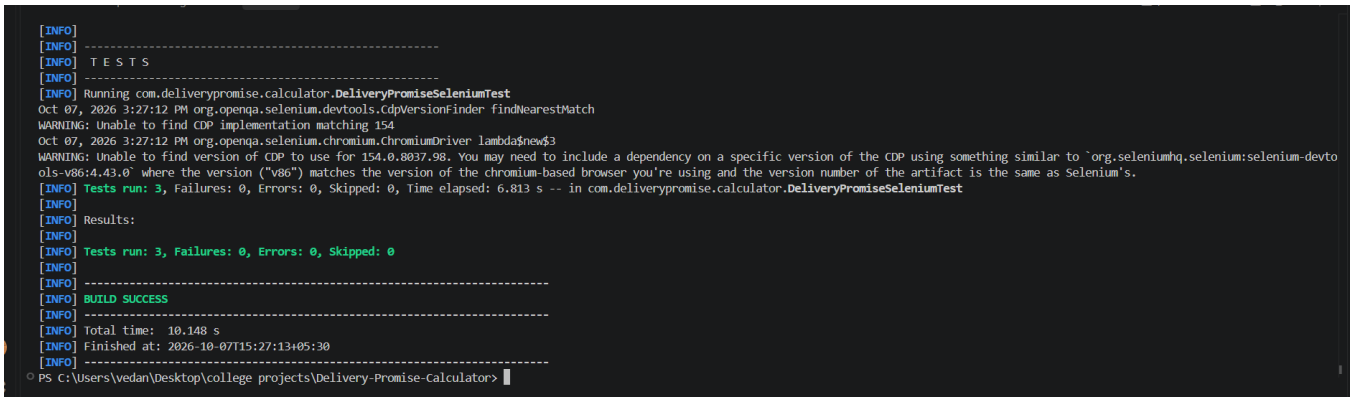
7. Testing Significance

The suite verifies behavior through a real browser, providing a functional end-to-end check rather than only unit-level validation.

Task 9 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Local Selenium execution: 3 tests, 0 failures, 0 errors, 0 skipped.



Task 10: Continuous Testing in Jenkins

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Integrate the Selenium browser tests into Jenkins so functional verification happens automatically as part of CI.

2. Integration

The Jenkins Maven Build and Test stage runs the project's test suite, including DeliveryPromiseSeleniumTest.

3. Git Workflow

The continuous-testing change was developed on feature/fix-jenkins-timeout. Commit ba55727 was created with message “Add Selenium continuous testing”. Because remote main was ahead, the feature branch was pushed and merged through a pull request.

4. Commands

```
git checkout -b feature/fix-jenkins-timeout
git add .
git commit -m "Add Selenium continuous testing"
git push -u origin feature/fix-jenkins-timeout
# PR: feature/fix-jenkins-timeout -> main
```

5. Jenkins Result

The Jenkins run reported 3 tests, 0 failures, 0 errors, 0 skipped, followed by BUILD SUCCESS.

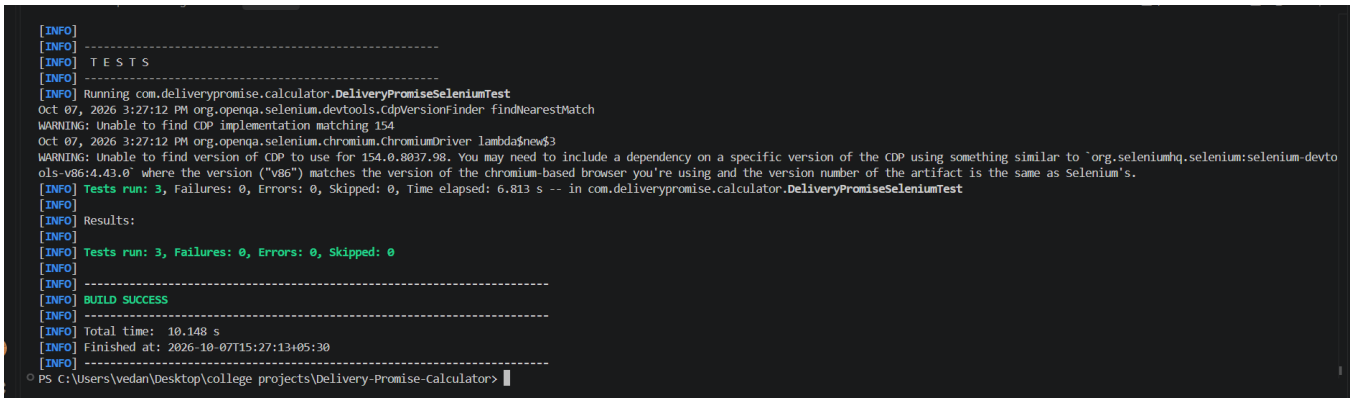
6. Result

Functional browser testing became part of the repeatable CI workflow and therefore acts as a gate before deployment.

Task 10 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Continuous test evidence: DeliveryPromiseSeleniumTest ran successfully with 3 passing tests.



Task 11: Docker Image and Container Lifecycle

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Containerize the Spring Boot application, build the Docker image, run a named container, and verify the application.

2. Dockerfile

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/delivery-promise-calculator-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8765

ENTRYPOINT ["java", "-jar", "app.jar"]
```

3. Build and Run

```
docker build -t delivery-promise-calculator:1.0 .
docker rm delivery-promise-app
docker run -d --name delivery-promise-app -p 8766:8765 delivery-promise-calculator:1.0
docker ps
```

4. Port Mapping

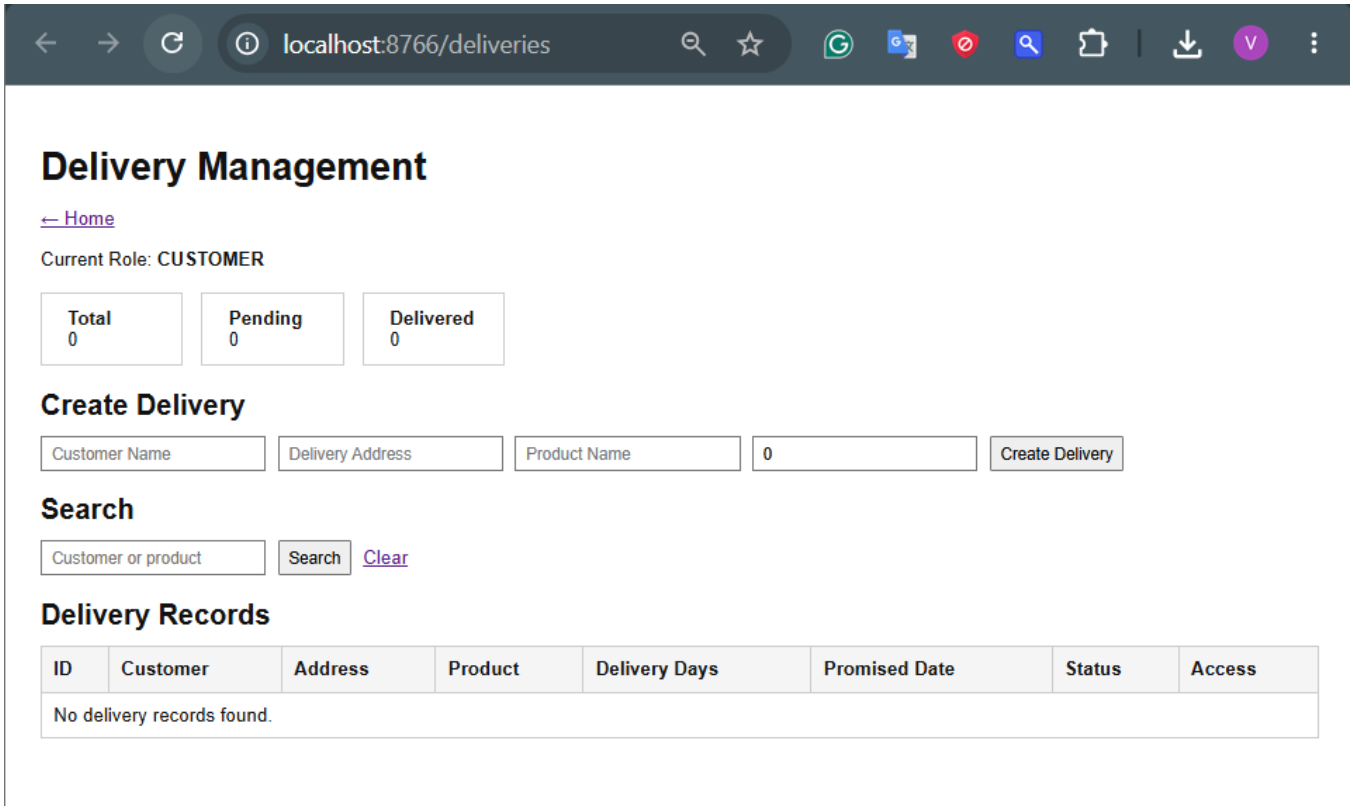
The application listens on 8765 inside the container. Host port 8766 was selected because the Jenkins-deployed application already occupied host port 8765.

5. Verification URL

```
http://localhost:8766/deliveries
```

6. Evidence

Evidence: Docker deployment accessed through the browser



7. Result

The Docker image and container were successfully built/run and the Delivery Management page loaded from the container.

Task 11 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Docker container/image verification from the command line.

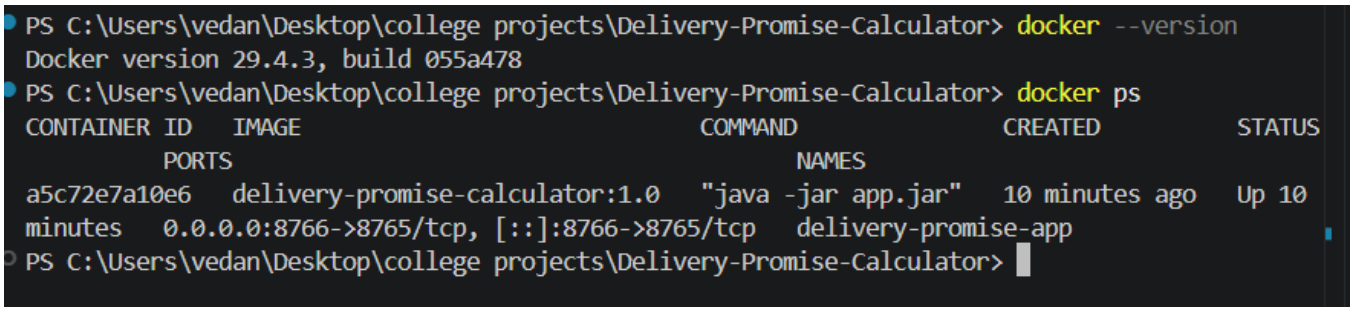
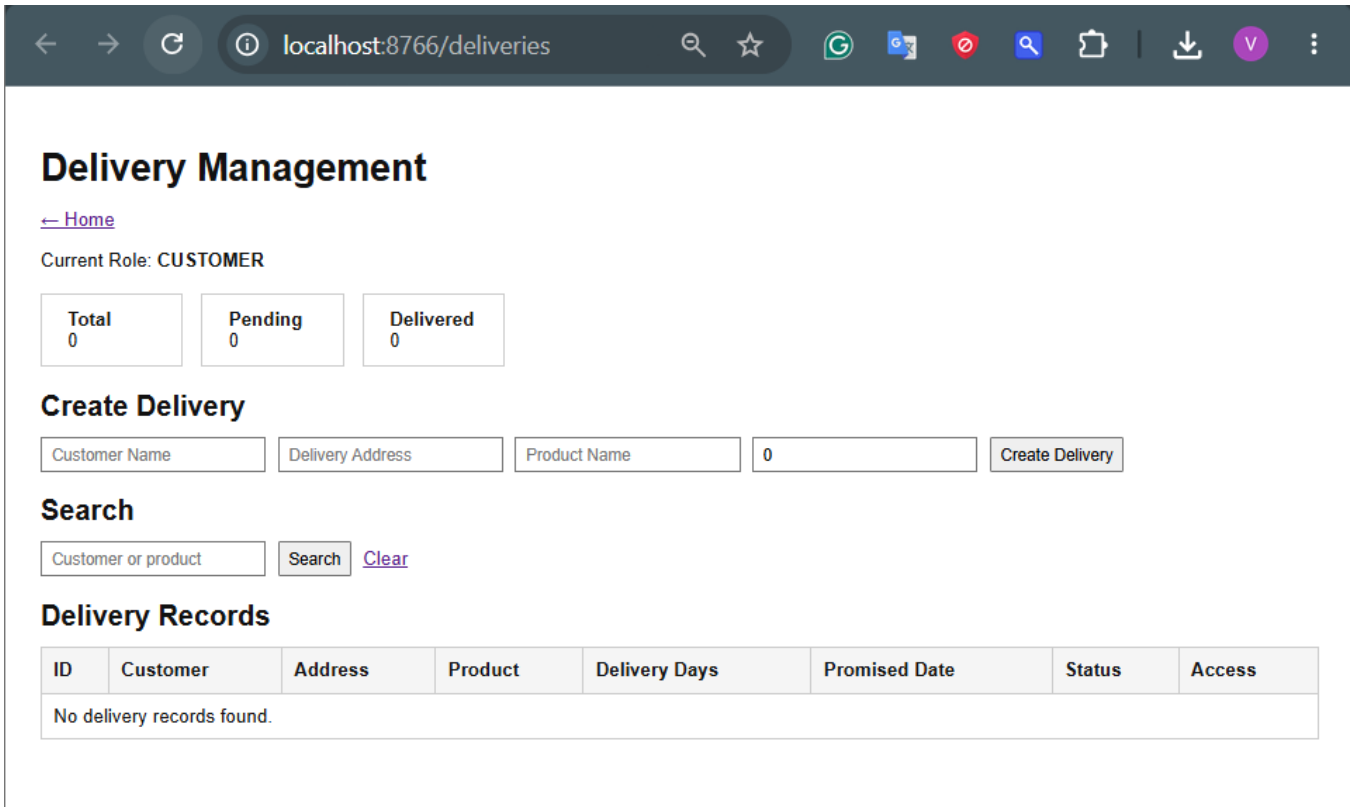


Figure 2. Delivery Management page served from the Docker deployment on localhost:8766.



Task 12: Jenkins Docker Continuous Deployment

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Integrate Docker image creation and container deployment into Jenkins so a successful build automatically produces and deploys the application container.

2. Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build and Test') {
            steps {
                bat '"C:\DevTools\apache-maven-3.9.16\bin\mvn.cmd" clean test package'
            }
        }

        stage('Docker Build') {
            steps {
                bat 'docker build -t delivery-promise-calculator:1.0 .'
            }
        }

        stage('Docker Deploy') {
            steps {
                bat '''
                docker rm -f delivery-promise-app 2>nul || exit /b 0
                docker run -d --name delivery-promise-app -p 8766:8765 delivery-promise-calculator:1.0
                '''
            }
        }

        stage('Verify Deployment') {
            steps {
                bat '''
```

```
                docker ps --filter "name=delivery-promise-app"
                echo Docker deployment completed successfully.
                '''
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully. Docker application deployed on port 8766.'
        }
        failure {
            echo 'Pipeline failed. Check the stage logs.'
        }
    }
}
```

3. Dockerfile Checkout Problem

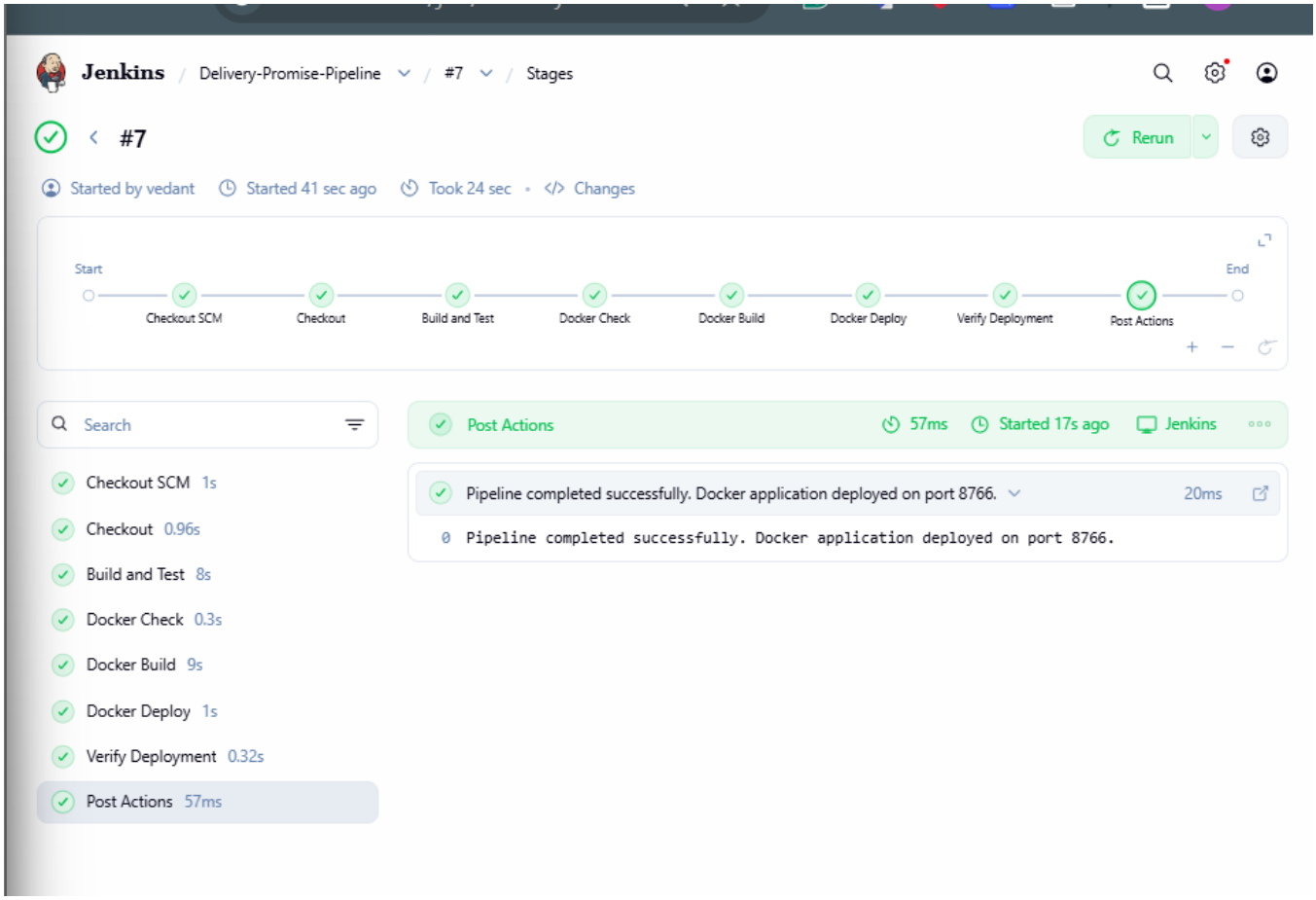
The first Docker build failed because the Dockerfile was not present in the Jenkins checkout of main. The Dockerfile was committed through the feature-branch/PR workflow and merged into main. The pipeline was then rerun.

4. Successful Pipeline

Build #7 passed Checkout SCM, Checkout, Build and Test, Docker Check, Docker Build, Docker Deploy, Verify Deployment, and Post Actions.

5. Evidence

Evidence: successful Jenkins-Docker deployment pipeline



6. Deployment Verification

The Docker application was accessed at http://localhost:8766/deliveries.

7. Result

Continuous deployment was achieved: Jenkins builds/tests the project, builds the image, replaces the container, starts the new container, and verifies the deployment.

Task 12 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Final Jenkins-Docker CD pipeline build #7: every stage passed and deployment was reported on port 8766.

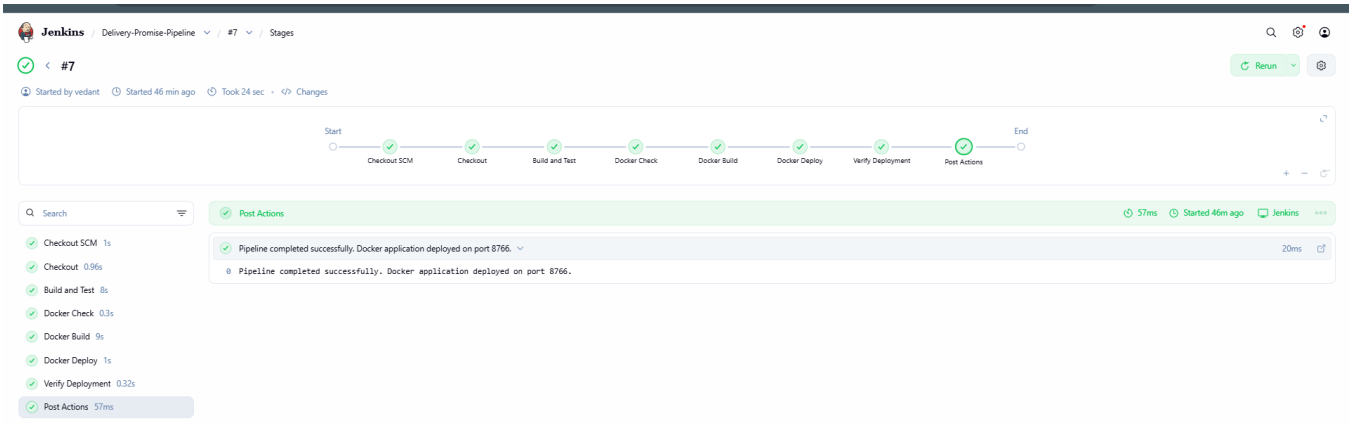
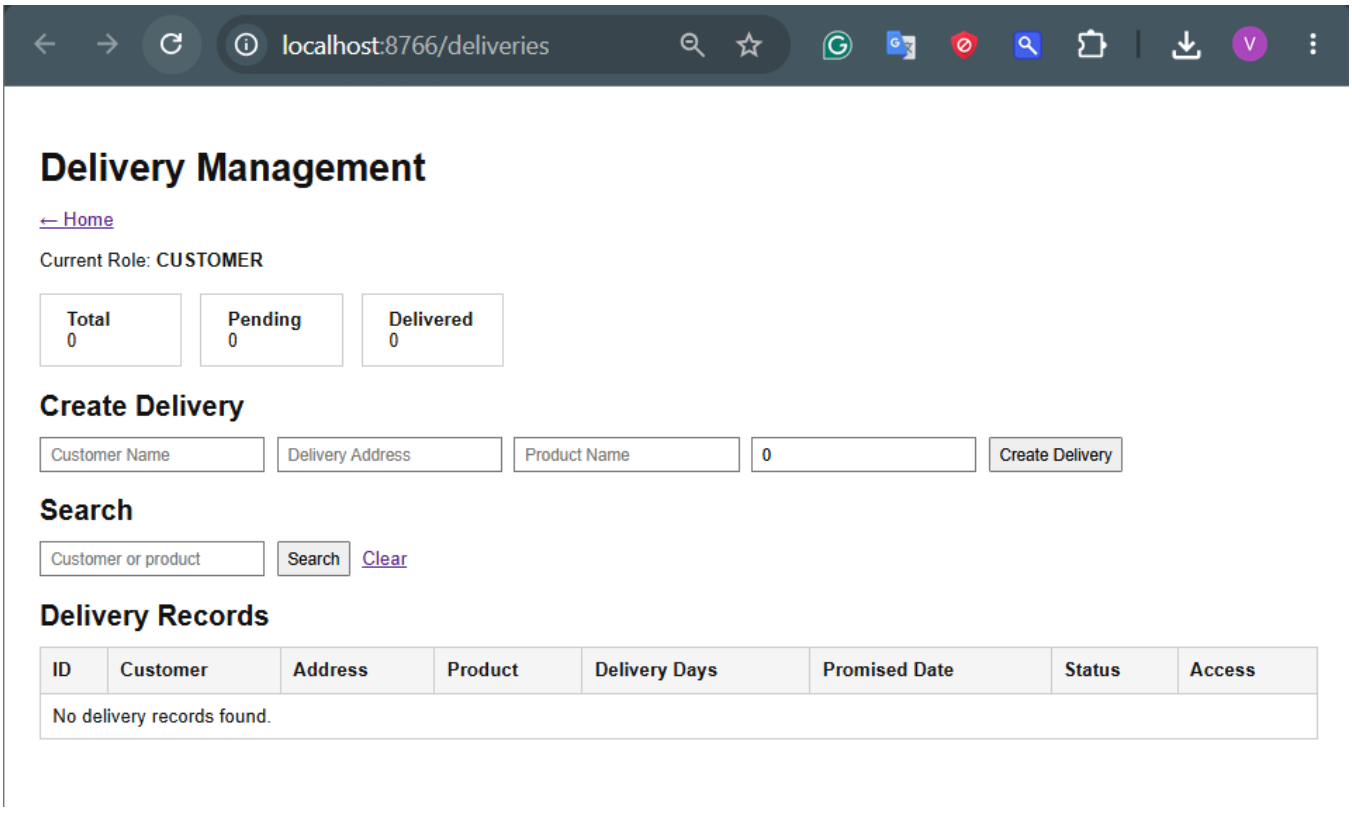


Figure 2. Browser verification of the containerized application at localhost:8766.



Task 13: Configuration Management using Puppet

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Use Puppet to define and maintain the desired configuration of the Delivery Promise Calculator deployment environment.

2. Puppet Installation

```
puppet --version
# 8.10.0

puppet config print vardir
# C:/Users/vedan/.puppetlabs/opt/puppet/cache
```

3. Puppet Manifest

```
# Task 13 - Delivery Promise Calculator
# Configuration management using Puppet

$delivery_root = 'C:/DeliveryPromise'

file { $delivery_root:
  ensure => directory,
}

file { "${delivery_root}/config":
  ensure  => directory,
  require => File[$delivery_root],
}

file { "${delivery_root}/logs":
  ensure  => directory,
  require => File[$delivery_root],
}

file { "${delivery_root}/config/application.properties":
  ensure  => file,
  content => @("PROPERTIES"),
spring.application.name=delivery-promise-calculator
server.port=8765
| PROPERTIES
  require => File["${delivery_root}/config"],
}
```

4. Apply and Verify

```
puppet apply puppet\delivery_promise.pp
Get-ChildItem C:\DeliveryPromise
Get-Content C:\DeliveryPromise\config\application.properties
```

5. Initial Result

Puppet created the config and logs directories and defined the application.properties content.

6. Idempotency Test

Applying the same manifest a second time produced only catalog compilation/application notices and no resource creation/change messages.

7. Git Integration

```
git checkout -b feature/puppet-configuration
git add puppet/delivery_promise.pp
git commit -m "Add Puppet configuration management"
git push -u origin feature/puppet-configuration
# PR -> main -> merge
```

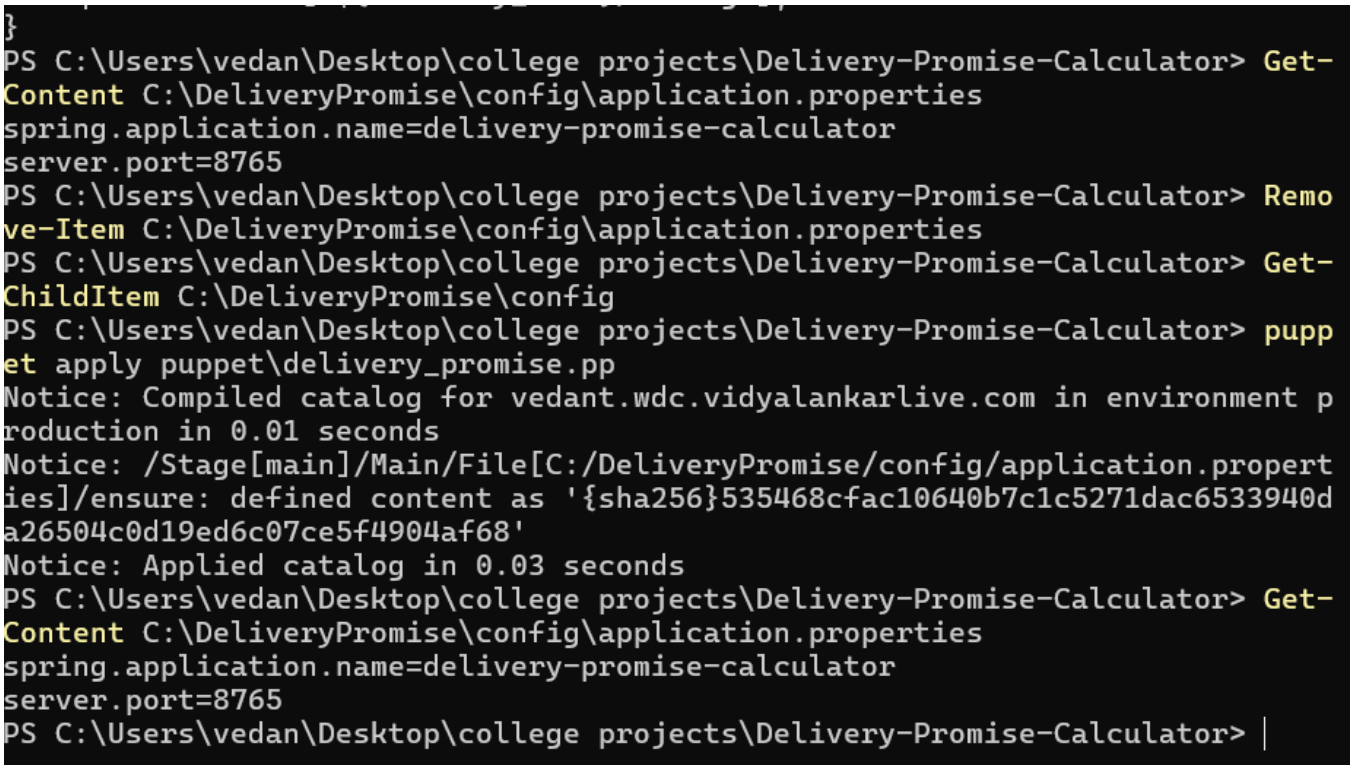
8. Result

Puppet successfully established a repeatable desired state and the manifest was merged into main.

Task 13 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Puppet manifest application and verification of the managed application.properties file.



Task 14: Automated Provisioning and Reliability

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Demonstrate automatic recovery when a managed configuration file is deleted or otherwise drifts from the desired state.

2. Baseline

```
Get-Content C:\DeliveryPromise\config\application.properties

spring.application.name=delivery-promise-calculator
server.port=8765
```

3. Simulate Drift

```
Remove-Item C:\DeliveryPromise\config\application.properties
Get-ChildItem C:\DeliveryPromise\config
```

4. Recover with Puppet

```
puppet apply puppet\delivery_promise.pp
```

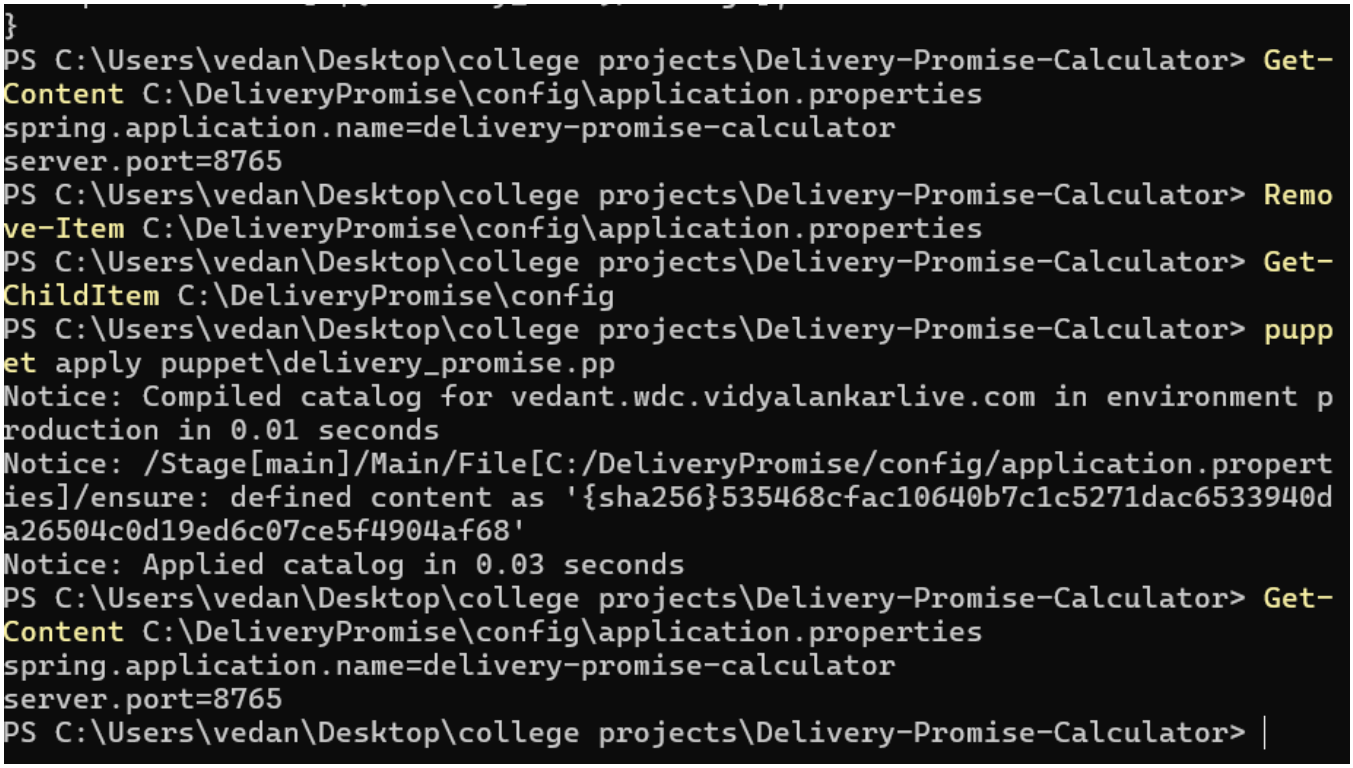
5. Verify Recovery

```
Get-Content C:\DeliveryPromise\config\application.properties

spring.application.name=delivery-promise-calculator
server.port=8765
```

6. Evidence

Evidence: configuration drift simulation, Puppet recovery, and verification



7. Reliability Findings

• Desired configuration is stored as code.

• Missing managed configuration is recreated automatically.

• Repeated application is idempotent.

• Recovery is repeatable and not dependent on manually recreating the file.

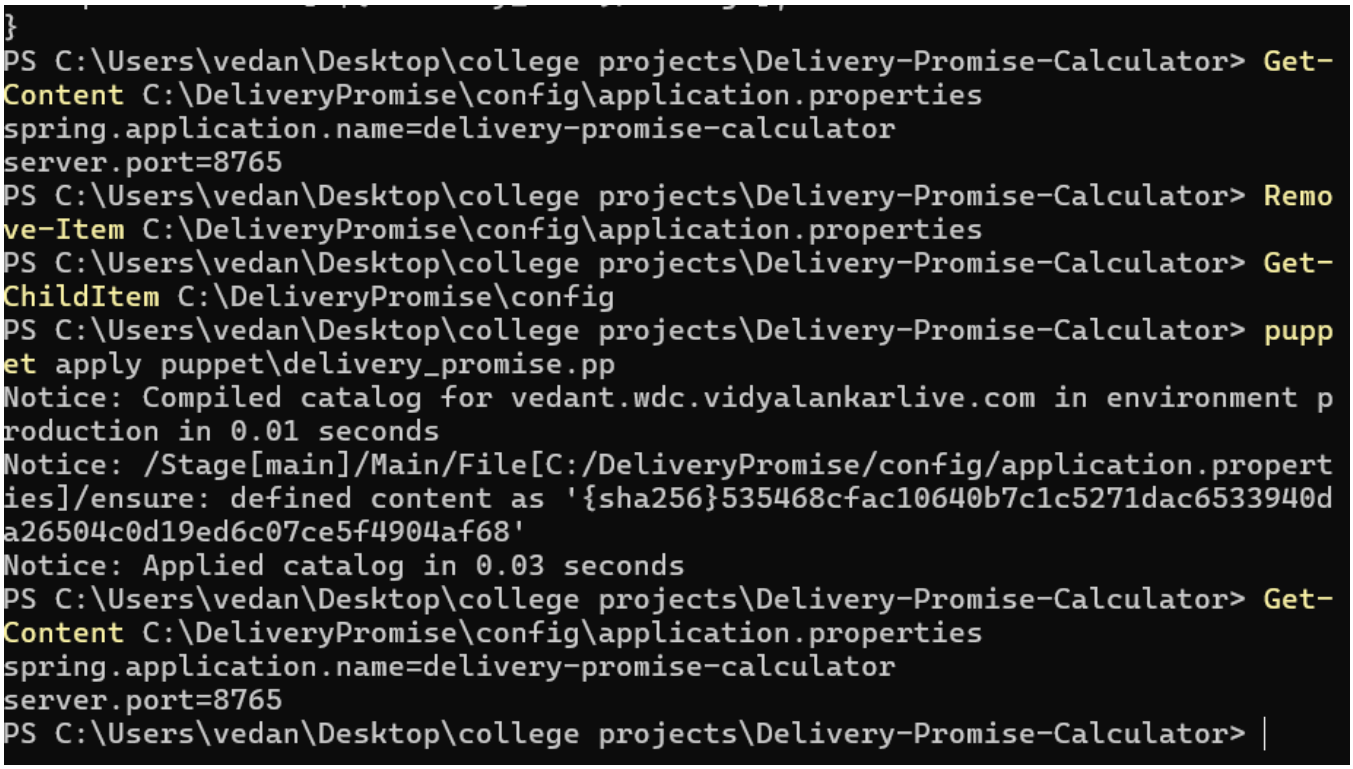
8. Result

The environment demonstrated automated provisioning/recovery after configuration drift.

Task 14 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Puppet re-applied the desired configuration and restored application.properties.



Task 15: Final End to End Release Documentation and Viva

Name: Vedant Rechwad **Roll No.:** 23102A0054

1. Objective

Document the final end-to-end DevOps release and provide concise viva preparation based on the implementation actually completed.

2. Final Workflow

```
Developer
  |
Git / GitHub
  |
Jenkins
  |
Maven Build + Automated Tests
  |
Docker Image
  |
Docker Container
  |
Host :8766 -> Container :8765
  |
Puppet-managed configuration
  |
Verified release
```

3. Final Technology Stack

Component	Project technology
Language/Runtime	Java 21
Framework	Spring Boot 4.1.1
Build	Maven 3.9.16

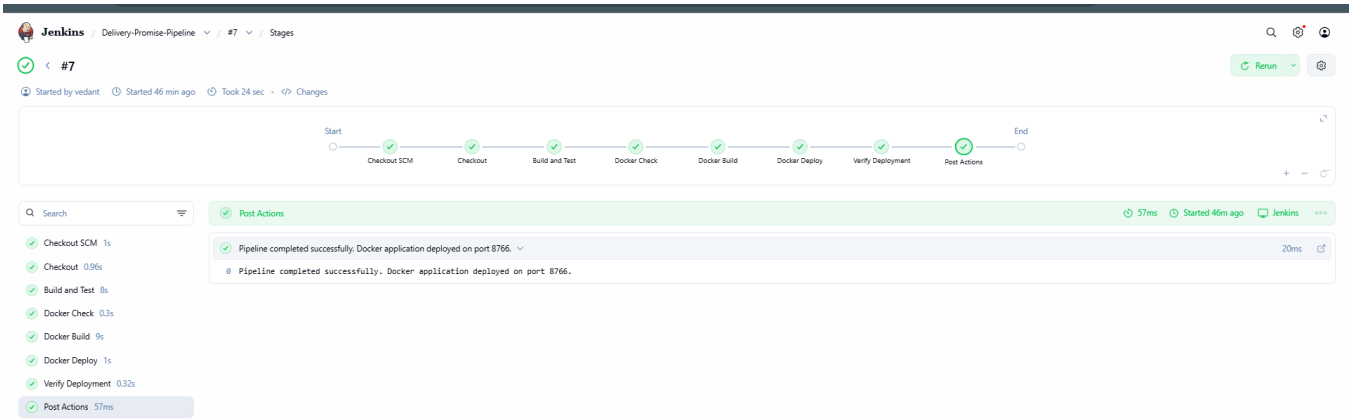
Database	SQLite + Spring Data JPA/Hibernate
UI	Thymeleaf + HTML/CSS
CI/CD	Jenkins
Testing	Selenium WebDriver + JUnit 5
Containerization	Docker
Configuration	Puppet 8.10.0
Version control	Git + GitHub

4. Final Jenkins Pipeline

The final pipeline includes Checkout SCM, Checkout, Build and Test, Docker Check, Docker Build, Docker Deploy, Verify Deployment, and Post Actions.

5. Final Evidence

Evidence: final successful Jenkins pipeline



6. Final Release Result

Jenkins build #7 completed successfully in 24 seconds. Every displayed stage passed and the post action confirmed Docker deployment on port 8766.

7. Testing

The Selenium suite previously completed with 3 tests, 0 failures, 0 errors, and 0 skipped tests and was integrated into the Jenkins build/test workflow.

8. Docker Deployment

The application is packaged with Eclipse Temurin 21 JRE. The container uses application port 8765 and host port 8766.

9. Puppet Reliability

Puppet 8.10.0 maintains C:/DeliveryPromise. Idempotency and recovery after configuration drift were both demonstrated.

10. Git Collaboration

Feature branches and pull requests were used throughout the project, including conflict resolution and the Puppet configuration merge.

11. Viva Questions

Question	Short answer
What is CI?	Automatically building/testing integrated changes.
What is CD?	Automatically deploying a successfully built release.
Why Docker?	Consistent, isolated and reproducible deployment.

Why port 8766?	Host 8765 was already occupied; container 8765 is mapped to host 8766.
What is Puppet?	Configuration management that enforces a desired state.
What is idempotency?	Repeated application reaches the same state without unnecessary changes.
Why Selenium?	Automated browser-level functional testing.
Why Jenkinsfile?	The CI/CD pipeline is stored and versioned as code.
What does Maven do?	Manages Java dependencies and build/test/package lifecycle.
What happens on failure?	Jenkins marks the stage/build failed and its logs are inspected.

12. Conclusion

The project demonstrates a complete DevOps chain: Git/GitHub for source control, Maven for Java build/test, Selenium for functional testing, Jenkins for CI/CD, Docker for containerized deployment, and Puppet for repeatable configuration management and recovery.

Task 15 — Actual Project Screenshots

Screenshots captured during the implementation are included below as evidence. These are the actual project screenshots, not recreated diagrams.

Figure 1. Final end-to-end Jenkins/Docker release evidence.

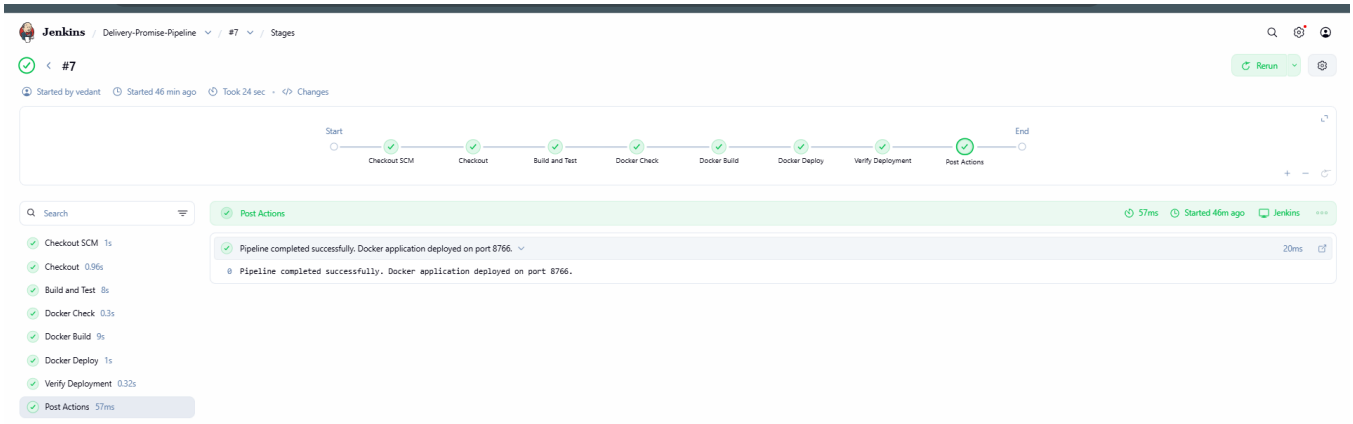


Figure 2. Final browser verification of the deployed application.

